CSC-239

Java Programming:

Chapter 5: Loops

and

Introduction to Error Handling

September 17, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder

- Quiz 1 began on Monday, September 15.
  - Online (Moodle) Quiz.
  - The Quiz will remain open until Monday September 22 , at 11:00 p.m.
  - Each student will have two hours to complete the Quiz, from the time that they begin.
- The material covered on Quiz 1 will be
  - The content of the course Syllabus
  - The lectures of September 3 and 8.
  - The quiz will include True / False, and Multiple-Choice questions.

Reminder: Submitting Files to *Moodle*

- When you submit your source code (*.java) files to *Moodle*, you MUST follow the **Submit Assignment to *Moodle*** procedure *completely*: be sure that the final Submission Status is:

  "Submitted for grading".



- This procedure is described in detail in the **Submit_Assignment_to_Moodle.pdf** document, which is available on *Moodle*, under the "**Handouts**" topic.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

While Loops

while (*loop-continuation-condition*) {

    statements(s);

}

Example:

```
int number = 1;
while(number <= 10) {
    System.out.println(number);
    number++;
}
```

Do-While Loop

```
do {
        statement(s);
} while (loop-continuation-condition);
```

- Similar to the “while” loop, but always executes at least once.

For Loop

for (*initial-action* ;

loop-continuation-condition;

action-after-each-iteration) {

statement(s);

}

- Usually used when the program knows how many iterations are needed.

Notes on `for` loops

- The initial-action in a for loop can be a list of zero or more comma-separated expressions.
- The action-after-each-iteration in a for loop can be a list of zero or more comma-separated statements. Therefore, the following for loop is correct.

```
for (int i = 0, j = 0; i + j < 10; i++, j++) {
  // Do something
}
```

Caution: “empty” loop body

- Adding a semicolon at the end of the for clause before the loop body is a common mistake, as shown below:

```
for (int i=0; i<10; i++);
{
   System.out.println("i is " + i);
}
```

Similar Cautions

- Similarly, the following loop is also wrong:

```
int i=0;
while (i < 10);
{
     System.out.println("i is " + i);
     i++;
}
```

Logic error: empty loop body

- In the case of the do loop, the semicolon **is** needed after the loop-continuation test, as shown below:

```
int i=0;
do {
     System.out.println("i is " + i);
     i++;
} while (i<10);
```

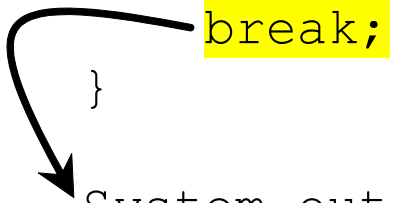
Correct

`break` Statement

```
public class TestBreak {
  public static void main(String[] args) {
    int sum = 0;
    int number = 0;

    while (number < 20) {
      number++;
      sum += number;
      if (sum >= 100)
        break;
    }

    System.out.println("The number is " + number);
    System.out.println("The sum is " + sum);
  }
}
```



`continue` Statement

```
public class TestContinue {
  public static void main(String[] args) {
    int sum = 0;
    int number = 0;

    while (number < 20) {
      number++;
      if (number == 10 || number == 11)
        continue;
      sum += number;
    }

    System.out.println("The sum is " + sum);
  }
}
```



Runtime Errors

- The compiler can detect syntax errors and some logic errors, but certainly not all runtime errors.
- Some runtime errors are “fatal” errors: the JVM outputs some message and the program stops.
- Exception Handling provides a way to handle some runtime errors *within* the program, so execution can continue.

ConvertInput.java

```
import java.util.Scanner;
public class ConvertInput {
public static void main(String[] args) {
        String text = null;
        int value = 0;
        Scanner input = new Scanner(System.in);
        while (true) {
            System.out.printf("Enter Text:  ");
            text = input.nextLine();
            System.out.printf("text=%s\n",  text);
            value = Integer.parseInt(inputText);
            System.out.printf("value = %d\n", value);
        }
    } // (end main)
} // (end class ConvertInput)
```

ConvertWithTryCatch.java

```
import java.util.Scanner;
public class ConvertWithTryCatch {
public static void main(String[] args) {
        String inputText = null;
        int value = 0;
        Scanner input = new Scanner(System.in);
        while (true) {
            try {
                System.out.printf("Enter Text:  ");
                inputText = input.nextLine();
                value = Integer.parseInt(inputText);
                System.out.printf("value = %d\n", value);
            } catch (NumberFormatException ex) {
                System.out.printf("Cannot convert [%s].\n",
                        inputText);
            }
        }
    } // (end main)
} // (end class ConvertWithTryCatch)
```

Division by Zero – a Classic Example

- Dividing one integer value by another:

```
int x = 5, y = 0;
int z = x / y;
```

- If divisor is zero, the result is undefined.
  - The JVM posts a runtime exception:

    java.lang.ArithmeticException: / by zero

Interactive Example

- The next slide: `InputMismatchExceptionDemo.java`
- This program tries forever, until the user enters an integer value.
- The exception, (InputMismatchException) is thrown by the Scanner class to indicate that the token retrieved does not match the pattern for the expected type, or that the token is out of range for the expected type.

InputMismatchException Demo

```
public static void main(String[] args) {
 Scanner input = new Scanner(System.in);
 boolean continueInput = true;
 do {
  try {
    System.out.print("Enter an integer: ");
    int number = input.nextInt();
    System.out.println( "number=" + number);
    continueInput = false;
  } catch (InputMismatchException ex) {
    System.out.println("Try again. (" +
    "Incorrect input: integer required)");
     input.nextLine(); // DISCARD INPUT
    }
  } while (continueInput); }
```

Exceptions

- Errors caused by your program or external circumstances.
- Can be caught and handled by your program.
- Our textbook covers Exception Handling in Chapter 12.
  - Runtime exceptions are very common with most of the Java library classes.
  - I believe it is worth looking at Exceptions earlier than Chapter 12.

Checked Exceptions vs. Unchecked Exceptions

- For *Checked* Exceptions, the compiler requires either:
  - use a try-catch block, or
  - "declare" the exception.
- *Unchecked* Exceptions:
  - May represent circumstances which the caller cannot be expected to handle, such as an unrecoverable system failure.
  - May represent elementary programming errors, and requiring the code to handle them would make the code too difficult to read.
- See also:
  http://docs.oracle.com/javase/tutorial/essential/exceptions/runtime.html

Declaring Exceptions

Every method must state the types of checked exceptions it might throw. This is known as *declaring exceptions*.

```
public void myMethod()
    throws IOException


public void myMethod()
    throws IOException, OtherException
```

Throwing Exceptions

- When the program detects an error, the program can create an instance of an appropriate exception type and throw it.
- This is known as *throwing an exception*.

Throwing Exceptions Example

```
/** Set a new radius */
  public void setRadius(double newRadius)
      throws IllegalArgumentException {
    if (newRadius >= 0)
      radius =  newRadius;
    else
      throw new IllegalArgumentException(
        "Radius cannot be negative");
  }
```

Catching Exceptions

```
try {
   statements;   // (may throw exceptions)
}
catch (Exception1 exVar1) {
   handler for exception1;
}
catch (Exception2 exVar2) {
   handler for exception2;
}
...
catch (ExceptionN exVarN) {
   handler for exceptionN;
}
```

Rethrowing Exceptions

```
try {
   statements;
}
catch(TheException ex) {
   perform operations before exit;
   throw ex;
}
```

The `finally` Clause

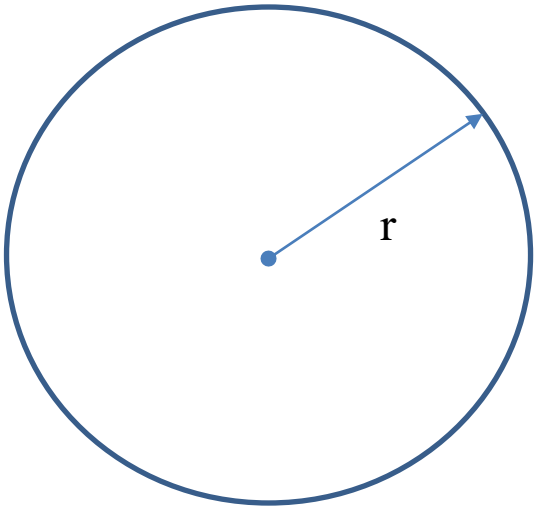
```
try {
   statements;
}
catch(TheException ex) {
   statements to handle ex;
}
finally {
   final statements;
}
```

Lab5a: Approximating π

A word of caution about this introduction:

- The next several slides use *mathematical* notation to express the concepts being discussed.
- In order to convert the *mathematical* statements into a program, you must understand what the mathematical statements mean.

The concept of π in Geometry



- The ratio of the circumference (perimiter) of a circle to its diameter is a constant called $\boldsymbol{\pi}$:

$$circumference = 2\pi r$$

- The area of a circle is also calculated using $\boldsymbol{\pi}$:

$$area = \pi r^2$$

π is an *irrational* number

- π cannot be represented *exactly* by any fraction or decimal representation.
- Any number that we use to represent π is only an *approximation*:
  - For practical purposes, an approximation is good enough, as long as we perform the calculation sufficient times .
  - It is possible to calculate π with whatever precision we need.

Lab5a: Approximate π

- The following *infinite series* represents a approximation for the geometry constant π:

$$\pi = 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \cdots + \frac{(sign)}{2i-1}\right)$$

where $(sign)$ alternates between the values of
+1 *and* **-1**.

The value ***i*** is a count of the terms in the series, starting at one.

- Math books usually display several terms of the series, and also the general formula to calculate as many terms as we want.

What the heck is an Infinite Series?

- Infinite Series:

  In mathematics, the sum of infinitely many numbers, whose relationship can typically be expressed as a formula or a function. An infinite series that results in a finite sum is said to *converge*.

  (Source: http://www.merriam-webster.com/dictionary/infinite%20series)

- We can choose to calculate as many terms of the series as we wish. If the series converges, then calculating more terms results in a more accurate approximation of the actual value.

Let's Take a Closer Look

- Each term of the series is calculated using the same formula: $\frac{(\boldsymbol{sign})}{\boldsymbol{2i-1}}$. The expression below specifically show the first 6 terms, in addition to the general formula for term $\boldsymbol{i}$:

$$\pi = 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \cdots + \frac{(sign)}{2i-1}\right)$$

When $\boldsymbol{i} == 1$, $sign == +1$, so $\frac{(sign)}{2i-1}$ evaluates to $+1$

When $\boldsymbol{i} == 2$, $sign == -1$, so $\frac{(sign)}{2i-1}$ evaluates to $-\frac{1}{3}$

When $\boldsymbol{i} == 3$, $sign == +1$, so $\frac{(sign)}{2i-1}$ evaluates to $+\frac{1}{5}$

Formatting output in Lab05a

- Lab05a specifies a very specific format for the output data. There is always some amount of "trial and error" when formatting detailed output.
- The **System.out.printf()** method provides a way to get precise control over the format of text. For example:

`%d`	Decimal integer.
`%6d`	Decimal integer, minimum 6 columns, space-filled.
`%f`	Floating-point number.
`%13.10f`	Floating-point, minimum 13 columns, 10 decimal places, space fill.
`%+014.10f`	Floating-point, minimum 14 columns, 10 decimal places, zero fill, display + sign.

Summary

- Loops
  - **`while`** loops,
  - **`do-while`** loops,
  - **`for`** loops
- **`break`** and **`continue`** Statements
- Runtime Errors and Exceptions
  - **`try`**
  - **`catch`**
  - **`throw`**
  - **`finally`**
  - **`Throws`**